Homework

Homework

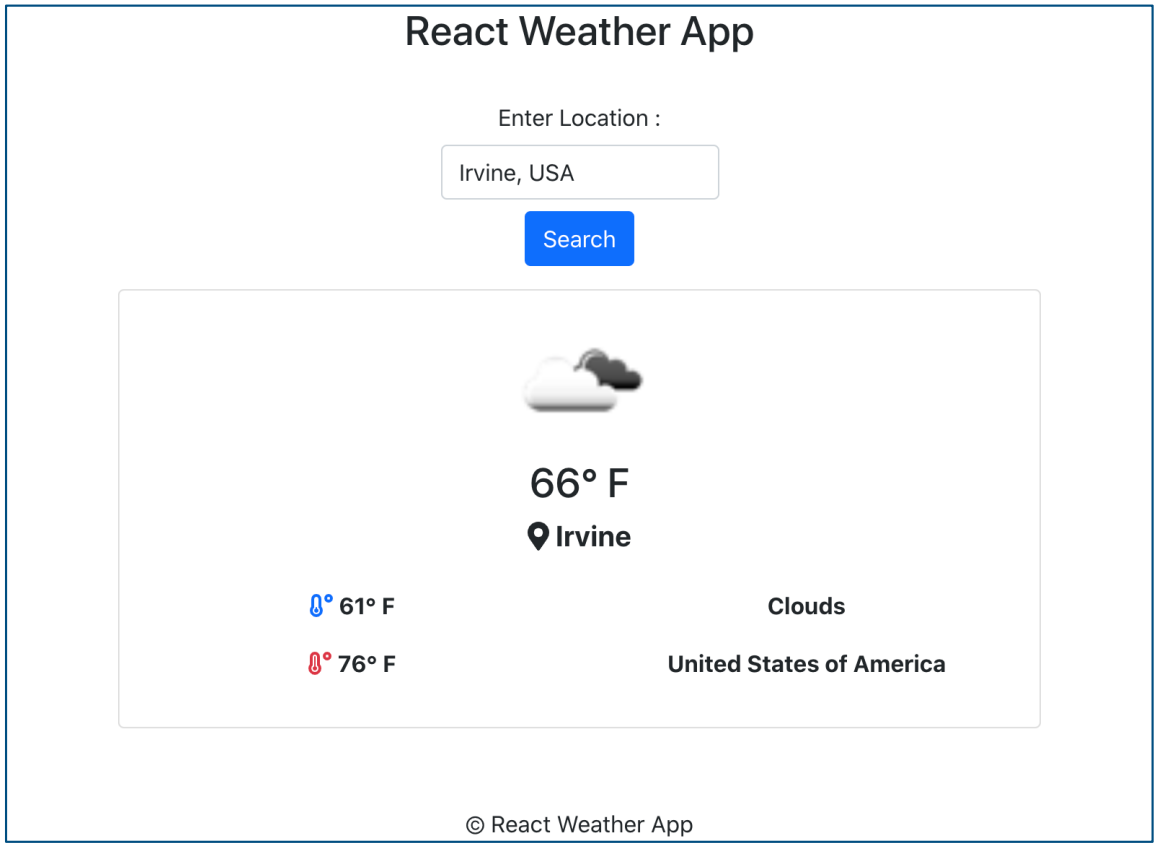
- For this homework, you will build a weather app using React, React-DOM and React-Ajax.
- To start, create a new Github repository **m5-w4-d1-homework** and then clone this in your WESTCLIFF folder.
- Then launch VS Code and navigate to the clone folder. Inside it, scaffold a react app:

  npx create-react-app react-weather-app

- Then navigate to the app folder: **cd react-weather-app**.
- Test the default app to make sure it works: **npm start**.

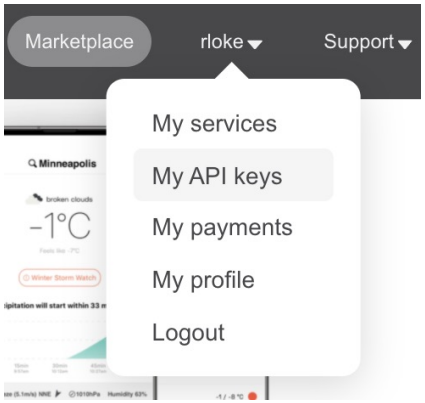
Homework

- Here's the weather app you will be building today:



Homework

- First, let's get the API Key from OpenWeatherMap API: https://openweathermap.org/.
- Create a new account and then login.
- After logged in, click your username on the top navigation bar, and select **My API Keys** from the dropdown:



- On the next screen, you should see your default key.

Homework

- You can generate a new API key anytime if you need to.
- Copy the API Key (you shouldn't share your API Key publicly).
- Let's return to the weather app - we'll store the generated API key in our project.
- Create a new file **.env** in the project's root folder (outside of the src folder).
- Add a new variable named REACT_APP_API_KEY= on line 1. Note: all API keys must be given the prefix REACT_APP.
- Then paste your key, like so:

  REACT_APP_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxx

- Next, install a country API called i18n-iso-countries to get countries dynamically. (https://www.npmjs.com/package/i18n-iso-countries)

  npm install i18n-iso-countries

Homework

- You will modify App.js next. Remove all the codes in there. Start by importing the following:

```
import { useState, useEffect } from 'react';
import './App.css';
import countries from 'i18n-iso-countries';
```

- Now add the following below the imports. Since you are using the package in a browser environment, you have to register the languages you want to use to minimize the file size:

```
countries.registerLocale(require('i18n-iso-countries/langs/en.json'));
```

- Below that, create the App component as a function with the standard return and export:

```
function App() {
    return (

    )
}

export default App;
```

Homework

- Now let's create some states to store the API data and input:

```
function App() {
    // State
    const [apiData, setApiData] = useState({});
    const [getState, setGetState] = useState('Irvive, USA');
    const [state, setState] = useState('Irvine, USA');
    return (
```

- Here's what these states do:
  - **apiData** state to store the response
  - **getState** to store the location name from input field
  - **state** to store a copy of the getState - this will be helpful in updating state on button click. You can ignore this state and directly pass getState on the URL as well.

Homework

- Below the useStates, create variables to store the API Key and URL:

```
const [state, setState] = useState('Irvine, USA');

// API KEY AND URL
const apiKey = process.env.REACT_APP_API_KEY;
const apiUrl = `https://api.openweathermap.org/data/2.5/weather?q=${state}&appid=${apiKey}`;
return (
```

- What these variables do:
  - **apiKey** to store the API key created earlier in the .env file.
  - **apiURL** to store the URL that calls the location from getState and also apiKey.

Homework

- Below that, you'll make an api request using Ajax fetch method and also the useEffect Hook. useEffect hook is used to perform side effects on your app. This lifecycle hook from react class components is an alternative to componentDidMount, unmount, etc:

```
// Side effect
useEffect(() => {
    fetch(apiUrl)
    .then((res) => res.json())
    .then((data) => setApiData(data));
}, [apiUrl]);
return (
```

- What this does is, it fetches the data from the given api url and stores in apiData state. This happens only when apiUrl changes thus it will prevent unwanted re-render. [ ] is the dependency array that determines when to re-render a component, when it is left empty it'll render only once. When we specify a dependency it will render only when it is updated.

Homework

- Below this, you'll write some functions to handle the input:

```
const inputHandler = (event) => {
  setGetState(event.target.value);
};

const submitHandler = () => {
  setState(getState);
};

const kelvinToFarenheit = (k) => {
  return ((k - 273.15)* 1.8 +32).toFixed(0);
};

return (
```

- **inputHandler**: this function is to handle the input field, to get the data and store in getState.
- **submitHandler**: this function is to copy the state from getState to state.
- **kelvinToFarenheit**: this function is to convert kelvin to celcius to farenheit with no decimals, and since we get the data from api as kelvin, we'll need to use this function to convert.

Homework

- Before proceeding to the next scripts, let's install bootstrap:

  npm install bootstrap

- And font awesome…

  npm i --save @fortawesome/fontawesome-svg-core
  npm install --save @fortawesome/free-solid-svg-icons
  npm install --save @fortawesome/react-fontawesome

- And now the imports:

```
import "bootstrap/dist/css/bootstrap.min.css";
import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";
import { faTemperatureLow, faTemperatureHigh, faMapMarkerAlt } from "@fortawesome/free-solid-svg-icons";
```

Homework

- Next, within the return(), create the following JSX structure:

```
return (
<div className="App">
    <header className="d-flex justify-content-center align-items-center">
        <h2>React Weather App</h2>
    </header>
    <div className="container">

    </div>
    <footer className="footer">
        &copy; React Weather App
    </footer>
</div>
)
```

Homework

- And within the container, there will be a form: a label with an input text box and a search button. We'll create a jsx element to wrap this form. And we'll start with the label:

```
<div className="container">
    <div className="mt-3 d-flex flex-column justify-content-center align-items-center">
        <div class="col-auto">
            <label for="location-name" class="col-form-label">
            Enter Location :
            </label>
        </div>
    </div>
</div>
```

Homework

- Below the label, we'll add the input text box:

```
    Enter Location :
    </label>
</div>
<div class="col-auto">
    <input
    type="text"
    id="location-name"
    class="form-control"
    onChange={inputHandler}
    value={getState}
    />
</div>
```

- And below the input, we'll add the button:

```
    <button
        className="btn btn-primary mt-2"
        onClick={submitHandler}
    >
        Search
    </button>
</div>
```

Note: the form-control is a bootstrap class that help formats form elements easier.
https://getbootstrap.com/docs/4.0/components/forms/

Homework

- Below the form wrapper, we'll create the second section that will contain the weather information. We'll start with a wrapper to contain the information:

```
    </div>

    <div className="card mt-3 mx-auto">

    </div>
</div>
<footer className="footer">
```

Homework

- Within this wrapper, you will create a ternary operator to test this condition:
  - if data is coming in from open weather based on user input location, then load and display data, otherwise, output a string message.

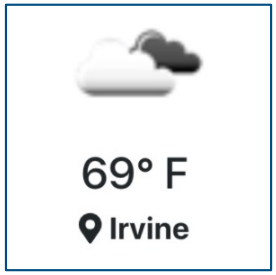
```
<div className="card mt-3 mx-auto">
    {/* Is it true data coming in from open weather based on input location */}
    {apiData.main
    ? (<div class="card-body text-center"></div>) /* Load data if true */
    : (<h1>Loading....</h1>)} {/* Outout message if false */}
</div>
```

Note: *apiData.main* attempts to extract weather data from open weather in the main category: https://openweathermap.org/current

```
],
"base": "stations",
"main": {
  "temp": 282.55,
  "feels_like": 281.86,
  "temp_min": 280.37,
  "temp_max": 284.26,
  "pressure": 1023,
  "humidity": 100
},
"visibility": 16093,
```

Homework

- If there's data coming in, we want data to be displayed. Here's the first batch of data we want to display – a status weather icon, current temp and location:



- Let's start with the icon:

```
{apiData.main
? (<div class="card-body text-center">
    <img
      src={`http://openweathermap.org/img/w/${apiData.weather[0].icon}.png`}
      alt="weather status icon"
      className="weather-icon"
    />
</div>) /* Load data if true */
```

Homework

- Right below the icon, the current temperature:

```
/>
<p className="h2">
    {kelvinToFarenheit(apiData.main.temp)}&deg; F
</p>
```

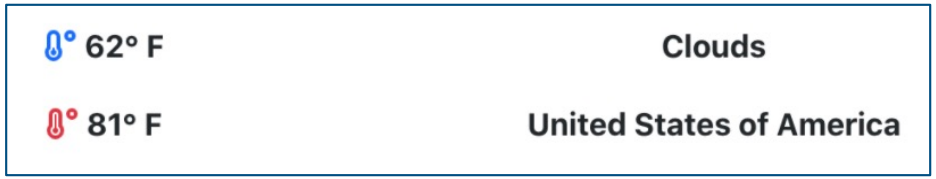
- And below current temp, user's location with a font awesome location icon:

```
    </p>
    <p className="h5">
        <FontAwesomeIcon
        icon={faMapMarkerAlt}
        className="fas fa-1x mr-2 text-dark"
        />{' '}
        <strong>{apiData.name}</strong>
    </p>
</div>) /* Load data if true */
```

```
},
"timezone": -25200,
"id": 420006353,
"name": "Mountain View",
"cod": 200
}
```

Homework

- The second batch data to display are: the day's low and high temperature, weather status and country:



- The first data are the low and high temp. But first, let's split this area into two equal columns with both wrapped in one single row element:

```
    </p>

    <div className="row mt-4">
        <div className="col-md-6"></div>
        <div className="col-md-6"></div>
    </div>
</div>) /* Load data if true */
```

Homework

- On the left column, we'll start with the low temp and a low temp font awesome icon:

```
<div className="col-md-6">
    <p>
        <FontAwesomeIcon
            icon={faTemperatureLow}
            className="fas fa-1x mr-2 text-primary"
        />{' '}
        <strong>
        {kelvinToFarenheit(apiData.main.temp_min)}&deg; F
        </strong>
    </p>
</div>
```

- Below that, the high temp and a high temp font awesome icon:

```
</p>
<p>
    <FontAwesomeIcon
        icon={faTemperatureHigh}
        className="fas fa-1x mr-2 text-danger"
    />{' '}
    <strong>
    {kelvinToFarenheit(apiData.main.temp_max)}&deg; F
    </strong>
</p>
```

Homework

- Moving to the right column, we'll start with the current weather status:

```
<div className="col-md-6">
    <p>
        {' '}
        <strong>{apiData.weather[0].main}</strong>
    </p>
</div>
```

- Below that, add country based on the user's input location:

```
    </p>
    <p>
        <strong>
        {' '}
        {countries.getName(apiData.sys.country, 'en', {
            select: 'official',
        })}
        </strong>
    </p>
</div>
```

Homework

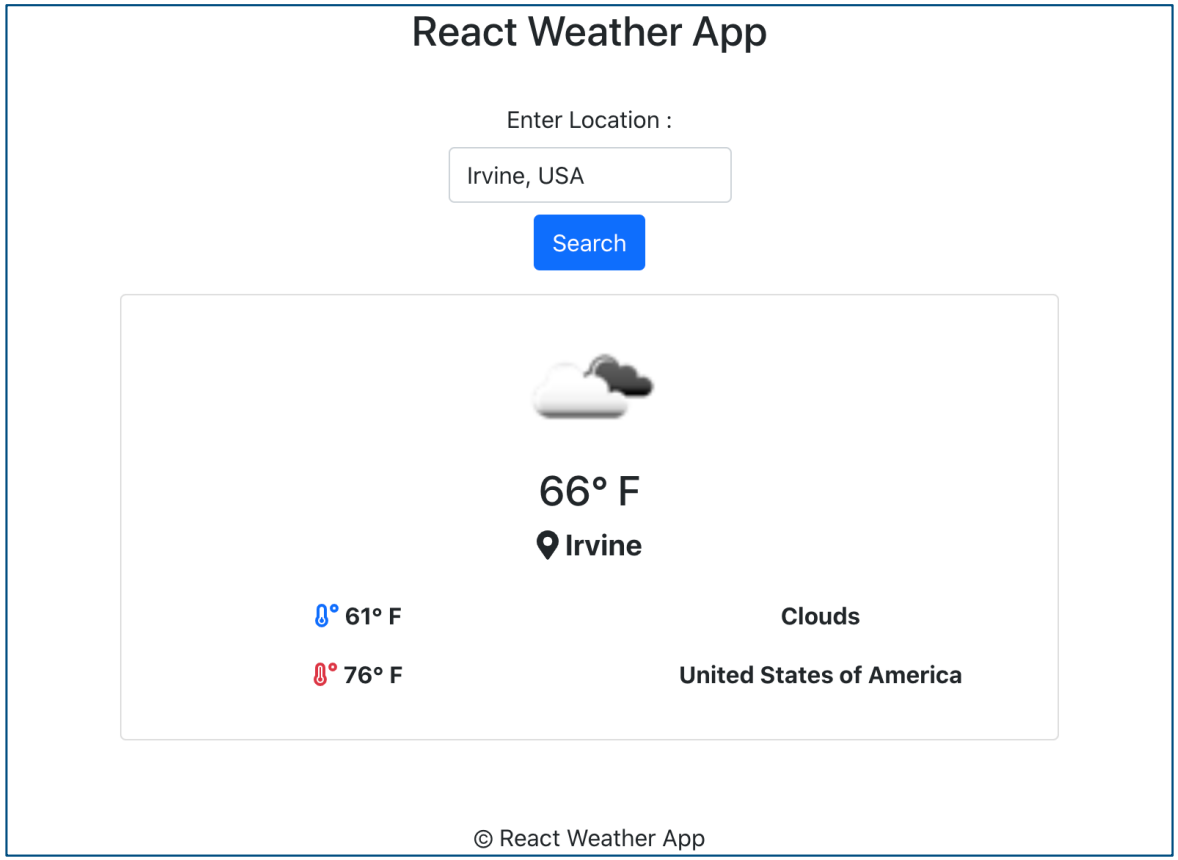
- And lastly, to add some final style touches to the app, add the following custom CSS in App.css:

```css
.weather-icon {
  width: 100px;
}

.container .card {
 width: 40em;
}

.footer {
  background: #fff;
  position: absolute;
  bottom: 0;
  left: 0;
  text-align: center;
  right: 0;
}
```

Homework

- Previewing and testing the app:

  npm start



Homework

- **DUE:**

  This Sunday, 10.30PM PT.

- **Submission:**

  Post Github link on GAP Week 4 Day 1 Homework

Connect with Us (WEB403/603/803)

Rich Loke

richloke@westcliff.edu

Elizabeth Kipp

elizabethkipp@westcliff.edu

Erin Tustin

erintustin@westcliff.edu

End of Presentation